

Análise Visual de Erros de Ponto Flutuante e das Units of Last Place em Experimentos Computacionais

Enzo Rocha Leite Diniz Ribas¹, D'Afonseca, L. A.², Rocha, L. M.³

Centro Federal de Educação Tecnológica de Minas Gerais, MG, Brasil

enzorochaleitedinizribas@gmail.com¹, luis.dafonseca@gmail.com², lucasmartinsrocha@gmail.com³

Introdução

Frequentemente, computadores realizam cálculos numéricos complexos que estão sujeitos a erros de arredondamento devido à quantidade finita de bits na representação computacional de números reais. Essa limitação agrava-se com o aumento da magnitude dos números, pois o espaçamento entre valores representáveis, a *Unit in the Last Place* (ULP), também cresce.

Neste trabalho, investiga-se, por meio de experimentos computacionais, os efeitos da representação em ponto flutuante sobre funções matematicamente exatas. Analisa-se a relação entre a escala dos valores, a resolução finita do sistema (IEEE 754) e a perda de significância. As simulações geram representações gráficas que permitem visualizar e identificar as regiões de oscilação onde a aritmética computacional diverge criticamente da aritmética exata, evidenciando o impacto da ULP no comportamento numérico.

Palavras-Chave: Erros Numéricos; IEEE 754; Métodos Numéricos; Cancelamento Catastrófico; Ponto Flutuante.

Fundamentação Teórica

Um Sistema de Ponto Flutuante (SPF) é a maneira utilizada para aproximar números reais pelos computadores através de representações binárias compactas. Para a maioria dos computadores atuais, o padrão adotado é o definido pela IEEE-754 [4].

No padrão IEEE 754 um número de ponto flutuante é dividido em três partes o **Sinal (S)**, o **expoente (E)** com viés (bias) e por fim, a **Mantissa (M)**. Definimos, então, a função $f(x)$ que leva um número real x a sua representação em ponto flutuante.

$$f(x) = (-1)^S \times (1.M) \times 2^{E-\text{bias}}, \quad (1)$$

A *Unit of Last Place* (ULP) é a menor diferença entre dois números representáveis em ponto flutuante, ou seja, a distância entre um número e o próximo número representável [1, 2]. O conjunto de números exatamente representáveis se torna mais esparsos quanto mais distante o número for de zero, aumentando o erro relativo. Dessa forma, a ULP se torna uma medida relativa importante para avaliar a precisão dos cálculos realizados com ponto flutuante.

Desenvolvimento e Resultados

Para analisar o comportamento das ULP's, foram realizados experimentos computacionais utilizando funções da forma $g(x, \ell) = x^{10} + \ell - x^{10}$, cujo valor exato é constante e igual a ℓ . Pela natureza binária do sistema de ponto flutuante, analisar diferentes potências de base 2 para ℓ revela uma boa visualização da grandeza da ULP(x) conforme x cresce.

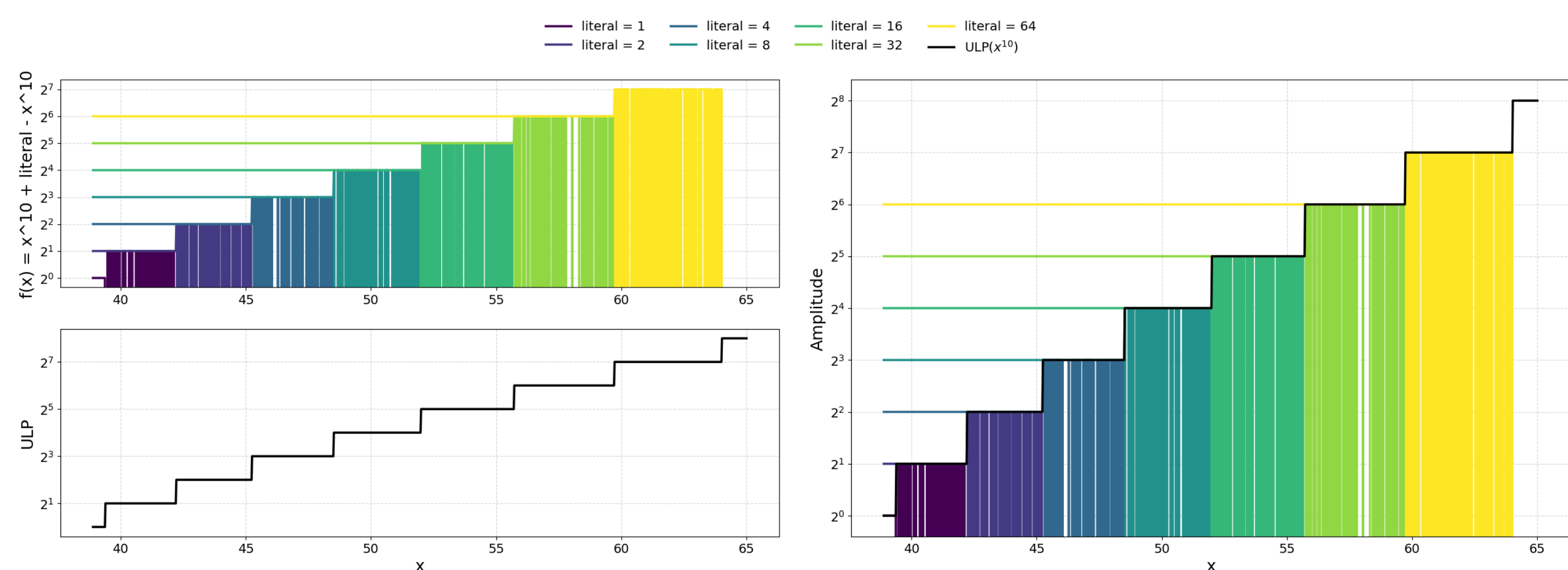


Figura 1: Resultado da avaliação da função $f(x) = x^{10} + l - x^{10}$ num SPF IEEE de 64 bits para $\ell = 2^n$, $n \in \mathbb{N}$, $n \leq 7$. Fonte: Autor.

Ao avaliar g para diferentes valores de ℓ , usando ponto flutuante, obtemos o gráfico da a ULP(x), ilustrado na Figura 1, em que o valor esperado $g(x, \ell) = \ell$ é exibido apenas até um certo x_1 , e a partir de um $x_2 > x_1$ a função assume o valor zero. Entretanto, no intervalo intermediário $[x_1, x_2]$ é observado uma oscilação aparentemente surpreendente, que tem amplitude igual a ULP(x^{10}) = 2ℓ .

Considere a função $f(x) = g(x, 1)$. O intervalo no qual ocorre a oscilação é aproximadamente $[39, 43]$ com uma amplitude de $2\ell = 2$, como observado no gráfico da Figura 2.

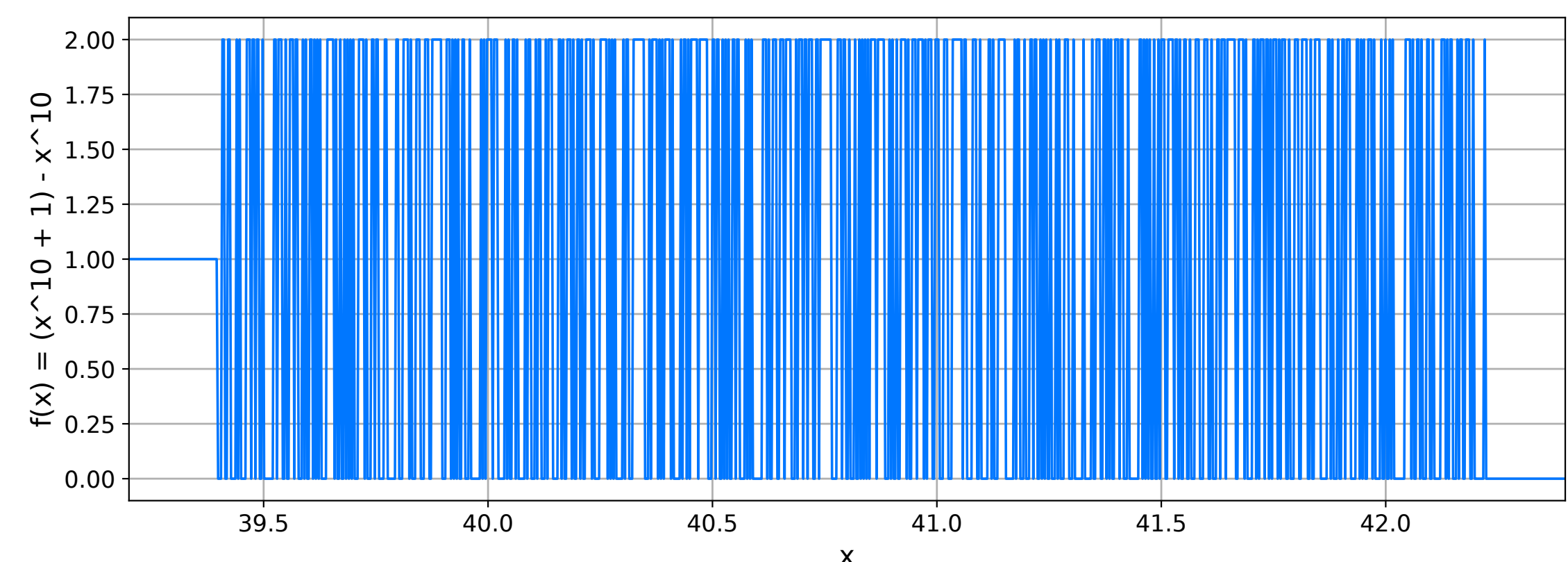


Figura 2: Resultado da avaliação da função $f(x) = x^{10} + 1 - x^{10}$ num SPF IEEE de 64 bits. Fonte: Autor.

Vamos analisar um exemplo em que $f(x) = 2$ para tentar compreender esse resultado inesperado. Considere $x = 42,1$. Na aritmética exata, a expressão avalia para $x^{10} = 17.491.254.479.245.335.8346502201$

Devido às limitações física do sistema, x não possui representação exata. O cálculo computacional inicia-se com a aproximação $f(x) \approx 42,10000000000000142 \dots$. Elevando este valor à décima potência, obtemos o valor flutuante A :

$$A = f(x) = (f(x))^{10} = 17.491.254.479.245.342$$

A estrutura binária de A no padrão IEEE 754 (64 bits) é dada por

0	100 0011 0100	1111 0001 0010 0001	1010 0000 0100 1111	0100 1010 1000 0000	1111
---	---------------	---------------------	---------------------	---------------------	------

Ao calcularmos a adição $B = A + 1$, o resultado exato (17.491.254.479.245.343) cai exatamente no meio de dois números de exatamente representáveis. O padrão IEEE 754 resolve esta ambiguidade utilizando a regra *Round to Nearest, Ties to Even* (arredondar para o mais próximo; em caso de empate, para o par, onde o bit mais a direita, chamado de *Least Significant Bit* (LSB) é nulo).

Analisando os dois candidatos à representação de B em binário, suprimindo o início para focar no final da mantissa temos:

Opção 1 - $(A) : [\dots]0100 1010 1000 0000 1111_2$

Opção 2 - $(A + \text{ULP}(A)) : [\dots]0100 1010 1000 0001 0000_2$

Como LSB da segunda opção é 0 (par), o sistema desempata arredondando para este valor. Logo, $f(B) = 17.491.254.479.245.344 = A + 2$.

Por fim, ao subtrair o termo x^{10} previamente calculado, temos a resolução final em máquina:

$$\begin{aligned} f(42, 1) &= f(x^{10}) + f(1) - f(x^{10}) \\ &= 17.491.254.479.245.344 - 17.491.254.479.245.342 = 2 \end{aligned}$$

Este salto de representação e arredondamento forçado fornecem a justificativa analítica exata para a oscilação verificada no intervalo intermediário.

Conclusões

Os experimentos computacionais conduzidos e a inspeção em baixo nível da estrutura binária provou que a anomalia visualizada não tem natureza estocástica, trata-se de uma consequência algorítmica e determinística das regras de arredondamento, especificamente o critério *Round to Nearest, Ties to Even*. O crescimento da ULP atua diretamente na amplitude desses erros, transformando operações analiticamente triviais em potenciais pontos de falha silenciosa no código.

Como desdobramentos lógicos para trabalhos futuros, propõem-se a avaliação do comportamento do cancelamento utilizando arquiteturas de dados emergentes, como `bfloat16` e *Posits* (Type III Unums), e o desenvolvimento de uma biblioteca (C/C++) para o monitoramento em tempo de execução da oscilação de ULPs em malhas de cálculo crítico, alertando sobre a perda de significância antes da consolidação do erro.

Referências

- [1] William Kahan. A logarithm too clever by half. *Unpublished note*, 2004.
- [2] Jean-Michel Muller. On the definition of ulp(x). *CNRS – Laboratoire LIP, projet Ainaire*, 2005.
- [3] Márcia Aparecida Gomes Ruggiero and Vera Lúcia da Rocha Lopes. *Cálculo numérico: aspectos teóricos e computacionais*. Pearson/Makron, Sao Paulo, 1998.
- [4] IEEE Computer Society. *IEEE Standard for Floating-Point Arithmetic*, 2019.